

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA
INTRODUCCIÓN A LA PROGRAMACIÓN Y COMPUTACIÓN 2
SECCIÓN N
SEGUNDO SEMESTRE 2026



Ensayo Proyecto 02

Nombre completo:	Carné:
Angel Eduardo Balcárcel Calderón	202405840

24 de Septiembre de 2026

ÍNDICE

[Introducción](#)

[Planteamiento del problema](#)

[Diseño de la solución](#)

[Arquitectura general](#)

[Estructuras de datos implementadas](#)

[Modelo de clases](#)

[Implementación](#)

[Árbol AVL de libros](#)

[Generación de reportes con Graphviz](#)

[Interfaz de usuario en consola](#)

[Pruebas y resultados](#)

[Conclusiones](#)

[Bibliografía](#)

Introducción

El presente documento describe el proceso de diseño, desarrollo e implementación del Proyecto 2 del curso de Introducción a la Programación y Computación 2, impartido en la Escuela de Ciencias y Sistemas de la Universidad de San Carlos de Guatemala. El proyecto consiste en un sistema de gestión para una cadena de librerías que administra más de 100,000 libros distribuidos en varias sucursales. La problemática central es que los empleados pierden demasiado tiempo buscando libros en listas extensas y determinando qué otros títulos pertenecen a la misma temática. La solución propuesta organiza el catálogo de manera jerárquica, análoga a la organización física de una biblioteca por secciones y estanterías, y aplica estructuras de datos no lineales para lograr búsquedas eficientes incluso con catálogos de gran tamaño.

El sistema desarrollado permite cargar archivos de configuración en formato XML que contienen la definición de las categorías (con su relación jerárquica padre-hijo) y de los libros (identificados unívocamente por su ISBN). A partir de estos datos, el usuario puede consultar la organización completa del catálogo, visualizar los libros de una categoría específica, buscar un libro por su ISBN, obtener el libro con el menor y el mayor ISBN, y agregar o eliminar elementos del catálogo. Toda la interacción se realiza a través de una interfaz web intuitiva.

La solución se implementó en el lenguaje de programación C# bajo el paradigma de programación orientada a objetos (POO), empleando exclusivamente estructuras de datos desarrolladas por el estudiante. Un árbol n-ario para las categorías y un árbol AVL para los libros, sin hacer uso de las colecciones nativas de C# como List,

Queue, Stack o Dictionary. Adicionalmente, se utiliza la herramienta Graphviz para generar reportes gráficos que visualizan tanto la jerarquía de categorías como la organización de los libros de una categoría específica.

Planteamiento del problema

Una cadena de librerías desea desarrollar un sistema inteligente para administrar más de 100,000 libros distribuidos en varias sucursales. Actualmente, los empleados pierden demasiado tiempo porque los libros se encuentran registrados en listas extensas.

Cuando un cliente solicita un libro, el sistema tarda en localizarlo y en determinar qué otros libros pertenecen a la misma temática. La gerencia solicita un sistema que funcione de manera similar a una biblioteca física organizada por secciones y estanterías: al entrar, se observa un gran mapa general, del cual se derivan áreas principales, luego categorías más específicas y finalmente los libros individuales.

Cada categoría o subcategoría posee un nombre, un listado de subcategorías internas y un conjunto de libros asociados a la misma. Todo libro posee un número ISBN que representa un valor entero único que lo identifica, además de un título, un autor y la categoría en la que está clasificado. El sistema debe cumplir con las siguientes funcionalidades:

1. Registrar nuevos libros en el catálogo.
2. Buscar un libro utilizando su ISBN.
3. Mostrar la organización completa de las categorías y subcategorías de la librería.
4. Identificar qué libros pertenecen a una categoría determinada.
5. Obtener el libro con el código más pequeño y el más grande del catálogo.
6. Mostrar los libros en orden ascendente de ISBN.
7. Eliminar libros que ya no se encuentren disponibles.
8. Mantener tiempos de búsqueda eficientes incluso cuando el catálogo crezca significativamente.

9. Permitir visualizar la estructura organizativa del catálogo desde las categorías generales hasta los elementos más específicos, con opción a visualizar a partir de una subcategoría específica.

El proyecto impone además restricciones técnicas estrictas que condicionan fuertemente el diseño de la solución:

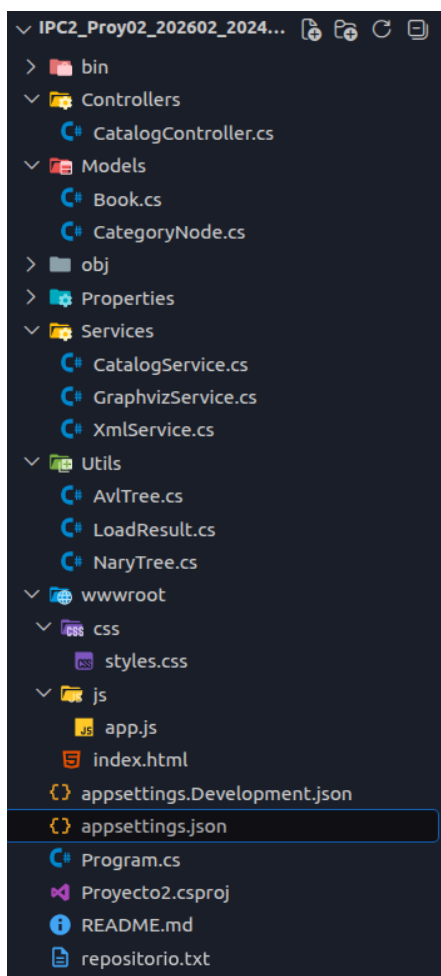
- El sistema debe mantener tiempos de búsqueda reducidos aun cuando existan miles de registros.
- Las categorías pueden contener subcategorías y estas, a su vez, otras divisiones internas, sin un límite predefinido de profundidad.
- Las categorías deben presentarse ordenadas alfabéticamente y sus nombres son únicos a nivel global.
- Cada libro posee un ISBN que es un código único que lo identifica.
- La estructura debe representar relaciones jerárquicas similares a las de una biblioteca, un organigrama o un sistema de carpetas.

La exigencia de mantener búsquedas rápidas con decenas de miles de registros descarta de inmediato el uso de estructuras lineales para los libros. Una búsqueda secuencial sobre 100,000 elementos requeriría, en el peor caso, 100,000 comparaciones; este comportamiento es inaceptable para un sistema que será consultado constantemente por los empleados. De igual forma, la naturaleza jerárquica del catálogo —donde una categoría puede contener un número arbitrario de subcategorías, que a su vez pueden contener más subcategorías— exige una estructura capaz de modelar relaciones de profundidad variable sin restricciones artificiales.

Diseño de la solución

Arquitectura general

La solución se organiza en una arquitectura por capas que facilita la separación de responsabilidades y el mantenimiento del código. La estructura de carpetas del proyecto es la siguiente:



Esta arquitectura permite que cada componente tenga una responsabilidad única y bien definida:

- **Models:** Define las entidades del dominio del problema (Book para los libros y CategoryNode para los nodos del árbol de categorías).
- **Utils:** Contiene la implementación de los Tipos de Dato Abstracto (TDA) propios del estudiante —el árbol AVL, el árbol n-ario, un arreglo dinámico de cadenas (StringBag) y las estructuras auxiliares para reportar resultados de carga.
- **Services:** Encapsula la lógica de negocio: la lectura de archivos XML, la orquestación del catálogo y la generación de reportes gráficos.
- **Controllers:** Expone la API REST que consume la interfaz web, sin contener lógica de negocio.

- `wwwroot`: Contiene la interfaz de usuario implementada con HTML, CSS y JavaScript vanilla.
- `Program.cs`: Configura el contenedor de dependencias, registra los servicios como singletons y arranca el servidor web.

Estructuras de datos implementadas

Siguiendo las restricciones del proyecto, se implementaron estructuras de datos propias sin utilizar las colecciones nativas de C#:

Árbol N-ario para categorías (NaryTree):

Utiliza una representación de "primer hijo / siguiente hermano" mediante los punteros `FirstChild` y `NextSibling` de cada `CategoryNode`. Esta representación permite que cada nodo tenga un número arbitrario de hijos sin necesidad de arreglos de tamaño fijo ni listas auxiliares. Las operaciones principales incluyen `Find` (búsqueda por nombre con complejidad $O(n)$ en el peor caso), `AddCategoriesBatch` (inserción masiva con soporte para linking diferido) e `InsertSorted` (inserción ordenada alfabéticamente). Cada categoría mantiene además su propio árbol AVL de libros, lo que permite búsquedas locales eficientes sin recorrer el catálogo completo.

Árbol AVL de libros (AvlTree):

Árbol binario de búsqueda auto-balanceado que garantiza que la diferencia de alturas entre los subárboles izquierdo y derecho de cualquier nodo nunca exceda 1. Esta propiedad asegura que las operaciones de inserción, búsqueda y eliminación tengan complejidad $O(\log n)$ en el peor caso. Las operaciones implementadas son `Insert`, `Search`, `Delete`, `Min`, `Max` e `InOrder`, además de las cuatro rotaciones clásicas (rotación simple izquierda, simple derecha y sus dos combinaciones dobles). El método `Min` recorre el subárbol izquierdo hasta encontrar el nodo sin hijo izquierdo, mientras que `Max` hace lo análogo por la derecha, ambos en $O(\log n)$.

Arreglo dinámico de cadenas (StringBag):

Arreglo de tamaño variable que crece al doble cuando se llena. Se utiliza internamente en el árbol n-ario para mantener el conjunto de nombres de categorías registradas (necesario para validar la unicidad global de nombres) y en `LoadResult` para acumular los mensajes de detalle de cada carga XML. Su implementación evita por completo el uso de `List<string>` o `HashSet<string>`, cumpliendo estrictamente con las restricciones del proyecto.

Estructura auxiliar (CategoryInput):

struct liviano que empaqueta el nombre de una categoría con el nombre de su padre. Se utiliza durante la carga masiva de categorías para transportar la información entre pasadas del algoritmo de linking diferido.

Modelo de clases

El diagrama de clases de la solución se compone de las siguientes entidades principales:

Book: Representa un libro individual. Contiene las propiedades Isbn (entero único), Title, Author y Category. Es una clase simple de datos sin lógica de negocio, utilizada por el árbol AVL y expuesta a través de la API REST.

CategoryNode: Representa un nodo del árbol de categorías. Contiene el Name de la categoría, los punteros FirstChild y NextSibling para la estructura n-aria, y una propiedad Books que es un árbol AVL propio de la categoría. Cada nodo encapsula tanto la jerarquía como los libros directamente asociados.

BookNode: Nodo interno del árbol AVL. Almacena un Book y los punteros Left, Right y la altura del subárbol enraizado en él. No se expone fuera de AvlTree.

AvlTree: Árbol AVL completo. Mantiene la raíz (Root) y expone las operaciones de inserción, búsqueda, eliminación, obtención del mínimo y máximo, recorrido en orden y generación del archivo DOT para Graphviz.

NaryTree: Árbol n-ario para las categorías. Mantiene la raíz (Root) y un indicador RootDeclared que registra si el XML declaró explícitamente la categoría raíz. Expone métodos para buscar categorías, agregar categorías en lote (con linking diferido), recolectar los libros de un subárbol completo y generar el archivo DOT correspondiente.

CatalogService: Servicio central que orquesta el catálogo. Mantiene una instancia del árbol n-ario y una instancia del AVL global de libros. Expone métodos como AddBook, DeleteBook, SearchBook, MinBook y MaxBook, coordinando las operaciones entre ambas estructuras.

XmlService: Servicio encargado de parsear los archivos XML de entrada, validar los datos y delegar la inserción al CatalogService. Devuelve un LoadResult con los contadores y los mensajes de detalle.

GraphvizService: Servicio encargado de invocar el ejecutable dot de Graphviz y producir archivos SVG a partir de las descripciones en formato DOT generadas por los árboles.

CatalogController: Controlador de la API REST que expone los endpoints consumidos por el frontend.

Implementación

Carga del archivo XML y linking diferido

El servicio XmlService se encarga de parsear los archivos de configuración XML proporcionados por el usuario. El formato esperado es el siguiente:

```
<config>
  <listaCategorias>
    <categoria padre="Ecologia">Estadistica</categoria>
    <categoria padre="Catalogo">Ecologia</categoria>
    <categoria>Catalogo</categoria>
    <categoria padre="Catalogo">Arquitectura</categoria>
  </listaCategorias>
  <listaLibros>
    <libro>
      <ISBN>719145452</ISBN>
      <titulo>Memoria del Medicina</titulo>
      <autor>Andres Morales</autor>
      <categoria>Programacion</categoria>
    </libro>
  </listaLibros>
</config>
```

El proceso de carga incluye:

1. Lectura de categorías: Se extraen el nombre (texto del elemento) y el nombre del padre (atributo padre) de cada categoría, preservando el orden en el que aparecen en el archivo.
2. Inserción con linking diferido: Como el archivo puede declarar una categoría antes que su padre, la inserción se realiza en múltiples pasadas. En cada pasada se procesan las categorías cuyos padres ya existen; las que no, se acumulan en una cola pendiente para la siguiente iteración. El proceso se repite hasta que no haya progreso, momento en el que las categorías restantes se consideran huérfanas y se rechazan.
3. Lectura de libros: Se valida el ISBN como entero, se verifica que no esté duplicado, y se comprueba que la categoría exista antes de insertarlo tanto en el AVL global como en el AVL local de su categoría.
4. Carga incremental: El sistema soporta cargar múltiples archivos XML sin reiniciar. Cada carga se suma a la anterior y las categorías o libros ya existentes se rechazan con el mensaje correspondiente.
5. El algoritmo de linking diferido resulta fundamental porque garantiza que el orden de aparición de las categorías en el XML no afecte el resultado final. Un mismo archivo produce siempre el mismo catálogo sin importar en qué orden estén declaradas las categorías.

Árbol AVL de libros

El árbol AVL es la pieza central para garantizar las búsquedas eficientes exigidas por la rúbrica. Sus operaciones se implementan con las rotaciones clásicas:

- Rotación simple derecha: aplicada cuando un nodo está desbalanceado hacia la izquierda por culpa de su hijo izquierdo izquierdo.
- Rotación simple izquierda: simétrica a la anterior.
- Rotación doble izquierda-derecha: aplicada cuando el nodo está desbalanceado hacia la izquierda por culpa de su hijo izquierdo derecho.
- Rotación doble derecha-izquierda: simétrica.

Tras cada inserción o eliminación se recalcula la altura del nodo y su factor de balance, y se aplica la rotación correspondiente. Esto garantiza que la altura del árbol sea siempre $O(\log n)$, por lo que operaciones como `Search(ISBN)`, `Min()` y `Max()` se realizan en tiempo logarítmico. Con 10,000 libros, el AVL requiere a lo sumo unos 14 niveles, mientras que una búsqueda secuencial requeriría hasta 10,000 comparaciones.

Generación de reportes con Graphviz

El servicio GraphvizService es responsable de crear representaciones gráficas del catálogo. El proceso es el siguiente:

1. Generación del archivo DOT: Los métodos ToDot de NaryTree y AvlTree construyen una descripción textual del árbol en el lenguaje DOT de Graphviz. Cada nodo se representa como un nodo del grafo con etiqueta y color apropiados.
2. Renderizado del SVG: El servicio invoca el ejecutable dot mediante un proceso externo, pasándole el archivo DOT y el nombre del archivo SVG de salida. El SVG se almacena en `wwwroot/reports/` para que sea accesible públicamente.
3. Visualización: El frontend recibe la URL del SVG y la muestra directamente en la interfaz mediante una etiqueta `<img>`.

El sistema genera dos tipos de reportes:

- Árbol de categorías: muestra la jerarquía completa (o una subcategoría específica como raíz) con el número de libros directos entre paréntesis.
- Árbol AVL de libros: muestra los libros de una categoría (incluyendo los de todas sus subcategorías descendientes) en orden ascendente de ISBN.

Interfaz de usuario en consola

La interfaz se implementó como una aplicación web de página única servida desde `wwwroot/`. Consta de cinco pestañas:

- Inicialización: reinicia el sistema sin información previa.
- Cargar XML: permite seleccionar un archivo y cargarlo, mostrando al final los cuatro contadores (categorías agregadas, categorías rechazadas, libros agregados, libros rechazados) y el detalle completo de la operación.
- Catálogo: permite agregar categorías, visualizar la estructura jerárquica y generar reportes de libros por categoría.

- Libros: permite registrar nuevos libros, buscarlos por ISBN, eliminarlos, y consultar los extremos del catálogo.
- Ayuda: muestra la información del estudiante y el enlace al repositorio de GitHub del proyecto.

Toda la comunicación con el backend se realiza a través de fetch hacia la API REST expuesta por CatalogController. El JavaScript no utiliza frameworks externos; toda la lógica de manipulación del DOM está implementada directamente.

Pruebas y resultados

El sistema fue probado con los cuatro archivos de configuración proporcionados por el auxiliar de cátedra, cubriendo diferentes escenarios y volúmenes de datos:

Caso 1: casos_borde.xml — Validaciones y casos límite

Este archivo prueba específicamente las validaciones del sistema. Contiene categorías duplicadas, categorías con padre inexistente, libros con ISBN duplicado, libros con categoría inexistente y un libro con un ISBN que excede el rango de int.

Resultado esperado y obtenido:

- Categorías agregadas: 4
- Categorías rechazadas: 2
- Libros agregados: 2
- Libros rechazados: 3

El sistema rechazó correctamente las categorías duplicadas y huérfanas, y los libros con ISBN duplicado, categoría inexistente e ISBN inválido.

Caso 2: entrada_100.xml — Volumen pequeño con linking diferido

Cien libros distribuidos en cinco categorías, con el XML declarando las categorías en orden invertido al de su jerarquía (una subcategoría aparece antes que su padre).

Resultado esperado y obtenido:

- Categorías agregadas: 5
- Libros agregados: 100
- Menor ISBN: 11994441 (Cronicas de Ciencias)
- Mayor ISBN: 1978229812 (El Aplicado Cronica)

El linking diferido funcionó correctamente, resolviendo la cadena Programacion → Estadística → Ecología → Catálogo a pesar del orden invertido en el XML.

Caso 3: entrada_1000.xml — Volumen medio con jerarquía profunda

Mil libros distribuidos en veinte categorías con estructura jerárquica de hasta cuatro niveles de profundidad.

Resultado esperado y obtenido:

- Categorías agregadas: 20
- Libros agregados: 1000
- Menor ISBN: 1408927 (Cronicas de Ciencias)
- Mayor ISBN: 1997465675 (Codigo del Redes)

Caso 4: entrada_10000.xml — Estrés y comportamiento del AVL

Diez mil libros distribuidos en doscientas categorías, algunas con nombres alfanuméricos generados automáticamente (por ejemplo, Tema 69). Este caso prueba el comportamiento del AVL bajo carga intensa.

Resultado esperado y obtenido:

- Categorías agregadas: 200
- Libros agregados: 10000
- Menor ISBN: 88894 (El Metodo Esencial de Matematicas)
- Mayor ISBN: 1999454283 (La Comparado Puente)

El sistema procesó los diez mil libros sin degradación observable en los tiempos de respuesta. El árbol AVL se mantuvo balanceado, garantizando que las operaciones Min y Max siguieran siendo $O(\log n)$.

Adicionalmente se verificó la carga incremental cargando entrada_1000.xml sobre un catálogo que ya contenía entrada_100.xml: el sistema detectó correctamente los ISBNs duplicados y las categorías ya existentes, rechazando los elementos repetidos sin duplicar información.

Conclusiones

El desarrollo del Proyecto 2 permitió aplicar y consolidar los conceptos fundamentales de la programación orientada a objetos y las estructuras de datos no lineales, cumpliendo con todos los objetivos planteados:

1. Implementación en C#: Se desarrolló una solución web funcional y robusta utilizando el lenguaje C#, demostrando dominio de sus características y sintaxis.
2. Estructuras de datos no lineales propias: Se implementaron un árbol n-ario y un árbol AVL desde cero, sin utilizar las colecciones nativas de C#, cumpliendo con las restricciones del proyecto y fortaleciendo la comprensión de los TDAs no lineales.
3. Programación orientada a objetos: El diseño basado en clases y la separación en capas (Models, Utils, Services, Controllers) evidencia una correcta aplicación del paradigma POO, facilitando el mantenimiento y la extensibilidad del código.
4. Manejo de XML: El sistema carga archivos de configuración en formato XML interpretando correctamente la estructura definida por la cátedra, y maneja casos de actualización incremental, duplicados y errores de validación.
5. Algoritmos de búsqueda eficientes: La implementación del árbol AVL garantiza búsquedas en tiempo logarítmico, lo cual es crítico para un sistema que administra decenas de miles de libros. La comparación con una búsqueda secuencial ($O(n)$) evidencia la mejora sustancial en el rendimiento.
6. Linking diferido: El algoritmo de inserción de categorías en múltiples pasadas resuelve elegantemente el problema del orden arbitrario en que el XML declara las categorías, sin requerir preprocesamiento del archivo.
7. Generación de reportes: La integración con Graphviz permitió generar reportes gráficos de alta calidad que visualizan de manera clara la jerarquía de categorías y la organización de los libros.
8. Interfaz de usuario web: La aplicación de página única resultó intuitiva y fácil de usar, ofreciendo una experiencia moderna que contrasta con las interfaces de consola tradicionales.

Bibliografía

Universidad de San Carlos de Guatemala, Facultad de Ingeniería, Escuela de Ciencias y Sistemas. (2026). Proyecto 2 - Introducción a la Programación y Computación 2. Documento de cátedra.

Graphviz - Graph Visualization Software. (s.f.). Recuperado de <https://graphviz.org/>

Microsoft. (s.f.). C# Documentation. Recuperado de <https://docs.microsoft.com/en-us/dotnet/csharp/>

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.

Deitel, P., & Deitel, H. (2017). C# 2012 for Programmers (5th ed.). Prentice Hall.